

Università degli Studi di Napoli
Federico II

A.A. 2024-2025

Programmazione

Estratto

Questa dispensa raccoglie, riorganizza e spiega il materiale del corso di Programmazione tenuto dal professor *Francesco Isgrò* per il corso di laurea triennale in Informatica della Federico II di Napoli (slide dell'A.A. 2024-2025). Il linguaggio del corso è il **C**.

È pensata per chi **parte da zero**: non si dà per scontato di aver mai scritto una riga di codice, né di aver mai usato un terminale. Il primo capitolo accompagna passo passo nell'installazione di tutto ciò che serve (l'editor **Visual Studio Code**, il compilatore e le estensioni) su Windows, macOS e Linux; i successivi seguono l'ordine delle lezioni.

L'obiettivo è **capire**, non memorizzare: ogni regola è accompagnata dal suo *perché*, ogni programma è stato compilato ed eseguito e ne viene mostrato l'output reale, e ciò che accade *dentro* il computer (le variabili in memoria, i puntatori, le chiamate di funzione, i nodi di una lista) è disegnato in uno schema. Dove le slide contengono una svista, il testo la corregge e lo segnala.

Sono inclusi **soltanto i contenuti tecnici** presentati a lezione; le informazioni organizzative (orari, modalità d'esame, appelli) restano sulle slide e sui canali del docente, dove vengono aggiornate. Questo documento **non sostituisce** le lezioni, le slide e i libri di testo: ne è un complemento.

Nel testo ricorrono alcuni elementi grafici: riquadri **verdi** con approfondimenti, **gialli** con avvertenze ed errori tipici, **rossi** con errori gravi da evitare, riquadri **Definizione** e riquadri **In sintesi** che riassumono il ragionamento. Il codice è disegnato come una finestra di VS Code e, **accanto**, compare ciò che produce: il **terminale**, dove i comandi digitati sono in bianco e ciò che si scrive mentre il programma è in esecuzione è in **giallo**, oppure uno schema.

Redazione a cura di **Fabio Rosiello**

Indice

1 L'ambiente di lavoro 4

Che cosa ci serve 4
Linux, Windows o macOS? 5
Passo 1: installare Visual Studio Code 6
Scaricare il programma 6
Installazione su Windows 7
Installazione su macOS 7
Installazione su Linux 8
Passo 2: installare il compilatore 8
Windows: WSL e Ubuntu 8
Windows: creare l'utente di Ubuntu 9
Windows: installare il compilatore in Ubuntu 9
macOS: gli strumenti da riga di comando 10
Linux 11
Passo 3: le estensioni di VS Code 11
Windows: collegare VS Code a Ubuntu 12
Passo 4: la cartella del corso 12
Com'è fatta la finestra di VS Code 13
Creare un file 14
Il terminale 14
Il prompt 15
Cartelle e percorsi 15
I comandi essenziali 16
Trucchi che fanno risparmiare tempo 17
Il primo programma 18
Compilare ed eseguire 18
Perché `./`? 19
Gli errori di compilazione 19
E il pulsante "Run"? 20
Quando qualcosa non funziona 21
Riferimenti 21

L'AMBIENTE DI LAVORO

Prima di scrivere il primo programma bisogna preparare il proprio *laboratorio*: qualche programma da installare e un paio di abitudini da prendere. Questo capitolo accompagna passo passo dall'installazione alla prima esecuzione, su **Windows**, **macOS** e **Linux**. Ogni passo è spiegato: non serve sapere nulla di computer oltre ad accenderli e a scaricare un file da Internet.

Leggi solo la parte del tuo sistema operativo

Le istruzioni di installazione sono divise per sistema operativo. Segui quelle del tuo computer e salta le altre: dalla sezione *Il terminale* in poi le istruzioni tornano a essere le stesse per tutti.

CHE COSA CI SERVE

Un computer non capisce il C. Capisce soltanto il proprio **linguaggio macchina**: lunghe sequenze di numeri che il processore sa eseguire, una dopo l'altra. Noi invece scriveremo i programmi in C, un linguaggio fatto di parole inglesi e simboli, comprensibile a una persona. Fra noi e il processore servono quindi tre strumenti, che compaiono nella figura 1.1.

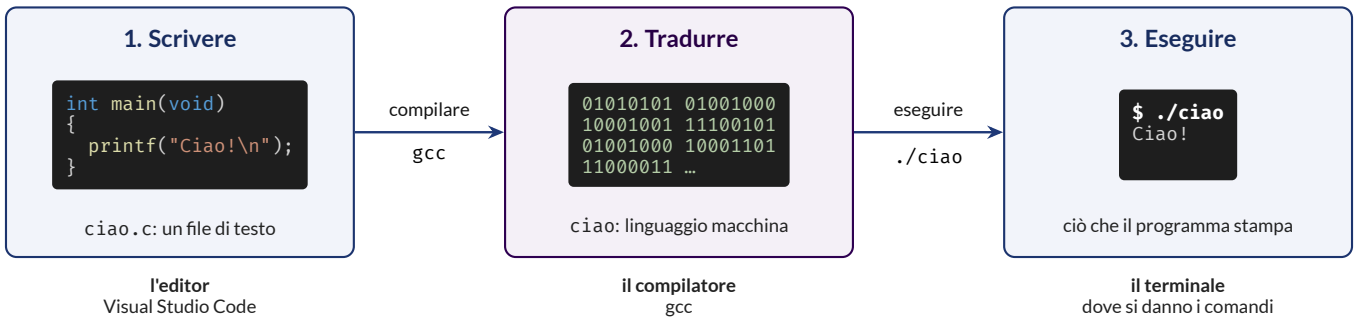


Figura 1.1: *Dal testo scritto da noi al programma che gira: i tre strumenti da installare.*

- L'**editor** è il quaderno su cui scriviamo il programma. Il programma, per ora, è un normale **file di testo** con estensione `.c` (per esempio `ciao.c`). Useremo **Visual Studio Code** (o semplicemente *VS Code*), un editor gratuito che colora il codice, segnala gli errori mentre si scrive e contiene al suo interno anche un terminale.
- Il **compilatore** è il traduttore: legge il file di testo e produce un nuovo file, l'**eseguibile**, che contiene lo stesso programma tradotto in linguaggio macchina. Useremo **gcc**, il compilatore C del progetto GNU, lo stesso usato a lezione.
- Il **terminale** è una finestra in cui si *scrivono* i comandi al computer invece di cliccare su icone. Da lì chiederemo a gcc di compilare e poi lanceremo il programma, che stamperà i suoi risultati nella stessa finestra.

Il capitolo **??** spiegherà in dettaglio che cosa succede durante la traduzione. Per ora basta l'idea: **si scrive, si compila, si esegue**.

Linux, Windows o macOS?

Il corso **assume Linux** come sistema operativo: i comandi visti a lezione (e all'esame) sono quelli di Linux. Non serve però cambiare computer o cancellare Windows: in ogni sistema c'è un modo semplice per avere gli stessi strumenti.

Sistema	Che cosa si installa	Perché
Windows	**WSL** (*Windows Subsystem for Linux*) con **Ubuntu**, e dentro Ubuntu gcc	WSL fa girare un vero Linux dentro Windows, senza riavvii né partizioni
macOS	gli **strumenti da riga di comando** di Apple (*Command Line Tools*)	macOS è un parente di Linux: ha già un terminale molto simile, manca solo il compilatore
Linux	il pacchetto `build-essential` (Ubuntu, Debian, Mint)	è il sistema del corso: basta un comando

VS Code si installa allo stesso modo ovunque (passo 1). Su Windows c'è una particolarità che conviene capire subito, perché spiega diversi passaggi successivi: i file del corso e il compilatore stanno *dentro* Ubuntu, mentre la finestra di VS Code è un normale programma di Windows. VS Code allora si divide in due parti che collaborano (figura 1.2): l'interfaccia, su Windows, e un piccolo *server*, dentro Ubuntu, che legge e salva i file e lancia il compilatore.

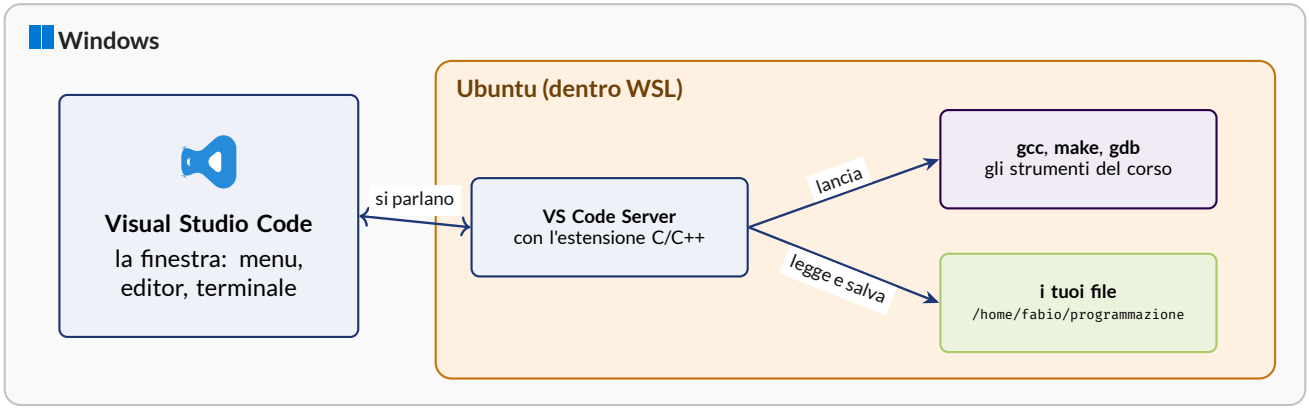


Figura 1.2: *Su Windows, con WSL: l'interfaccia di VS Code resta su Windows, mentre file e compilatore vivono in Ubuntu. Su macOS e su Linux tutto sta nello stesso sistema e questa divisione non c'è.*

Perché non un compilatore "nativo" per Windows?

Esistono versioni di gcc per Windows (MinGW, per esempio) e funzionano. Ma con esse alcuni dettagli cambiano: l'eseguibile si chiama `a.exe` invece di `a.out`, la fine dell'input si segnala con Ctrl + Z invece di Ctrl + D, i comandi del terminale sono diversi. Con WSL si usano **esattamente** i comandi delle lezioni, ed è ciò che conta.

Passo 1: installare Visual Studio Code

Scaricare il programma

Visual Studio Code si scarica gratuitamente dal sito ufficiale, `https://code.visualstudio.com`. La pagina *Download* propone un pulsante per ogni sistema operativo (figura 1.3): si sceglie il proprio.

Visual Studio Code, non Visual Studio

Microsoft produce anche **Visual Studio** (senza "Code"): è un altro programma, molto più grande, pensato per altri linguaggi. Il nome giusto è **Visual Studio Code**, con l'icona blu della figura 1.3.

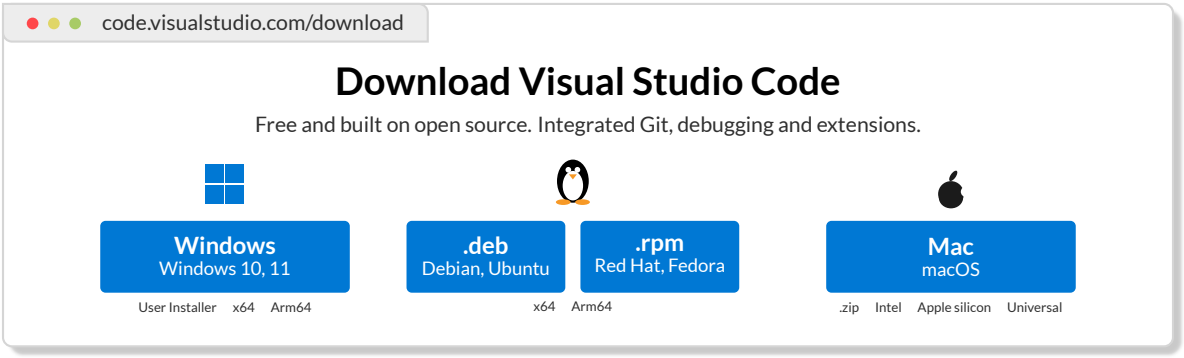


Figura 1.3: *La pagina di download di VS Code. L'aspetto del sito cambia spesso; i tre gruppi di pulsanti, invece, restano.*

Installazione su Windows

Il pulsante *Windows* scarica un file chiamato più o meno `VSCodeUserSetup-x64-1.xx.x.exe` (i numeri sono la versione). È un normale programma di installazione:

1. Apri il file scaricato (doppio clic, oppure clic sulla notifica di download del browser).
2. Nella prima schermata scegli *I accept the agreement* (accetti la licenza) e fai clic su *Next*.
3. Nelle due schermate successive (cartella di installazione e menu Start) lascia tutto com'è e fai clic su *Next*.
4. Nella schermata *Select Additional Tasks* (figura 1.4) spunta le due voci *Add "Open with Code" action…*: aggiungono la voce "Apri con Code" al menu del tasto destro sui file e sulle cartelle, comodissima. La voce **Add to PATH** deve restare spuntata: permette di aprire VS Code scrivendo `code` nel terminale, e la useremo.
5. Fai clic su *Next*, poi su *Install*. Al termine, lascia spuntato *Launch Visual Studio Code* e fai clic su *Finish*.

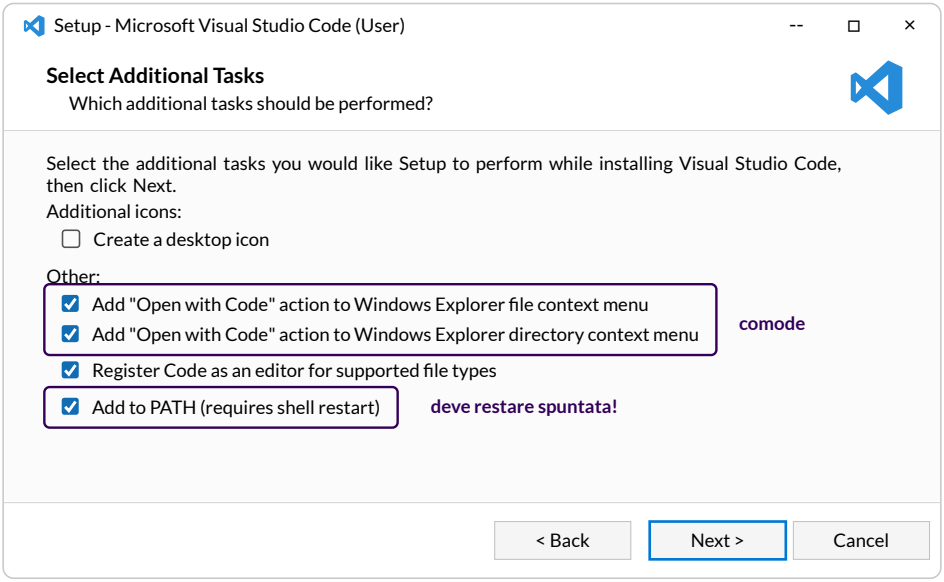


Figura 1.4: *La schermata più importante dell'installazione su Windows.*

Installazione su macOS

Il pulsante *Mac* scarica un archivio `.zip`. Safari di solito lo estrae da solo: nella cartella *Download* compare l'applicazione *Visual Studio Code* (altrimenti basta un doppio clic sul file `.zip`).

1. Trascina *Visual Studio Code* dalla cartella *Download* alla cartella *Applicazioni* (figura 1.5): così è installata davvero e la si trova nel Launchpad.
2. Aprila. Al primo avvio macOS avvisa che è un'applicazione scaricata da Internet: conferma con *Apri*.
3. Per poter aprire VS Code dal terminale con il comando `code`: premi Cmd + Shift + P, scrivi *shell command* e scegli *Shell Command: Install 'code' command in PATH*.

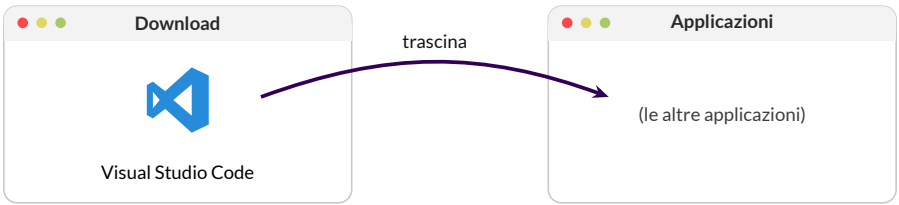


Figura 1.5: *Su macOS si installa un'applicazione trascinandola nella cartella Applicazioni.*

Installazione su Linux

Su Ubuntu, Debian e derivate si scarica il pacchetto `.deb` e lo si installa dal terminale (il terminale si apre con Ctrl + Alt + T; il suo uso è spiegato più avanti in questo capitolo):

Terminale

```
$ cd ~/Scaricati
$ sudo apt install ./code_*.deb
```

Il comando `cd` entra nella cartella dei download (su un sistema in inglese si chiama `Downloads`); `sudo apt install` installa il pacchetto, dopo aver chiesto la password dell'utente. In alternativa, VS Code si trova anche nel negozio di applicazioni della distribuzione (su Ubuntu, *App Center*).

PASSO 2: INSTALLARE IL COMPILATORE

Windows: WSL e Ubuntu

Su Windows 10 (aggiornato) e Windows 11 WSL si installa con un unico comando, da scrivere in **PowerShell** aperta **come amministratore**, cioè con il permesso di modificare il sistema.

1. Premi il tasto Windows, scrivi *powershell* e, nel riquadro a destra, scegli *Esegui come amministratore* (figura 1.6). Windows chiede conferma: rispondi *Sì*.
2. Nella finestra blu (o nera) che si apre, scrivi il comando `wsl --install` e premi Invio.
3. Attendi il download (qualche minuto: Ubuntu pesa circa un gigabyte) e, quando richiesto, **riavvia il computer**.

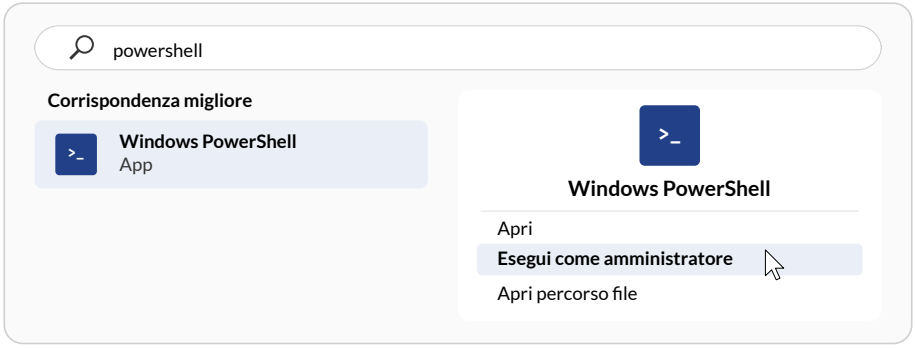


Figura 1.6: *La ricerca nel menu Start: PowerShell va aperta come amministratore.*

Amministratore: Windows PowerShell

```
PS C:\WINDOWS\system32> wsl --install
Installazione: Piattaforma macchina virtuale
Piattaforma macchina virtuale è stato installato.
Installazione: Sottosistema Windows per Linux
Sottosistema Windows per Linux è stato installato.
Installazione: Ubuntu
Ubuntu è stato installato.
L'operazione richiesta è riuscita. Le modifiche saranno effettive al riavvio del sistema.
```

Windows: creare l'utente di Ubuntu

I messaggi possono essere un po' diversi a seconda della versione di Windows: quello che conta è che non compaiano errori. Dopo il riavvio si apre da sola una finestra di Ubuntu (se non si apre, cerca *Ubuntu* nel menu Start) che completa l'installazione e chiede di creare un **utente Linux**: un nome (in minuscolo, senza spazi; non deve essere per forza uguale a quello di Windows) e una password.

Ubuntu

```
Installing, this may take a few minutes...
Please create a default UNIX user account. The username does not need to match your Windows username.
For more information visit: https://aka.ms/wslusers
Enter new UNIX username: fabio
New password:
Retype new password:
passwd: password updated successfully
Installation successful!
fabio@PC-FABIO:~$
```

Mentre scrivi la password non compare nulla

Nel terminale, mentre si digita una password, **non compaiono nemmeno gli asterischi**: sembra che la tastiera non funzioni, invece i caratteri vengono letti. Scrivi la password e premi Invio. Annotala: servirà ogni volta che installerai qualcosa.

Windows: installare il compilatore in Ubuntu

L'ultima riga stampata da Ubuntu, `fabio@PC-FABIO:~$`, è il **prompt**: Ubuntu è pronta a ricevere comandi (la spiegheremo pezzo per pezzo più avanti). Restano da installare il compilatore e gli altri strumenti del corso. Scrivi questi due comandi, uno alla volta, premendo Invio dopo ciascuno:

Ubuntu

```
$ sudo apt update
[sudo] password for fabio:
...
$ sudo apt install build-essential gdb
...
Do you want to continue? [Y/n] y
...
```

- `sudo` (*superuser do*) esegue il comando che segue con i permessi di amministratore, e per questo chiede la password appena creata.
- `apt` è il **gestore di pacchetti** di Ubuntu: una specie di negozio di applicazioni che si usa dal terminale. Con `update` scarica l'elenco aggiornato dei programmi, con `install` installa quelli indicati (e chiede conferma: si risponde `y`, *yes*).
- `build-essential` è un pacchetto che ne raccoglie altri: il compilatore **gcc**, il programma **make** (che useremo dal capitolo **??**) e le *librerie* standard del C. **gdb** è il *debugger*, utile per eseguire un programma un'istruzione alla volta.

Per verificare che tutto sia andato a buon fine, si chiede a gcc la sua versione:

Ubuntu

```
$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

I numeri di versione possono essere diversi: va bene qualunque versione. Se invece compare `gcc: command not found`, l'installazione non è riuscita: ripeti i due comandi `apt` e controlla che non stampino errori.

Se `wsl --install` non funziona

L'errore più comune all'avvio di Ubuntu è un messaggio che parla di *virtualizzazione* (per esempio il codice `0x80370102`): WSL usa una funzione del processore che su alcuni computer è disattivata. Si attiva dal BIOS/UEFI del computer (la voce si chiama *Virtualization Technology*, *Intel VT-x* o *AMD-V*); la procedura cambia da modello a modello, quindi conviene cercare "attivare virtualizzazione" seguito dal modello del proprio PC. Le istruzioni ufficiali di Microsoft sono indicate nei riferimenti a fine capitolo.

macOS: gli strumenti da riga di comando

Su macOS il compilatore fa parte degli *strumenti da riga di comando* di Apple, gratuiti. Non serve installare Xcode (l'ambiente di sviluppo completo, molto più grande): bastano gli strumenti.

1. Apri il **Terminale**: premi Cmd + Spazio (si apre Spotlight), scrivi *terminale* e premi Invio.
2. Scrivi il comando `xcode-select --install` e premi Invio.
3. Compare una finestra come quella della figura 1.7: fai clic su *Installa* (non su *Ottieni Xcode*) e accetta la licenza. Il download richiede qualche minuto.

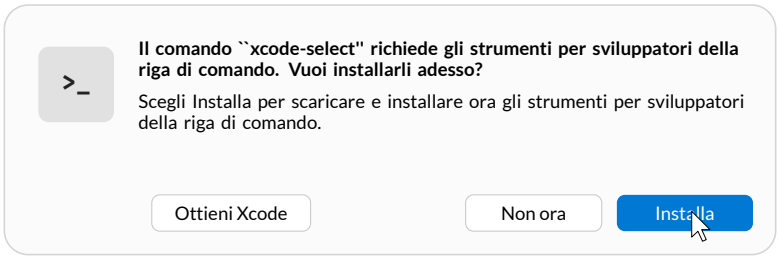


Figura 1.7: *La richiesta di installazione degli strumenti da riga di comando (il testo esatto dipende dalla versione di macOS).*

Al termine, la verifica:

Terminale di macOS

```
$ gcc --version
Apple clang version 16.0.0 (clang-1600.0.26.6)
Target: arm64-apple-darwin24.1.0
Thread model: posix
InstalledDir: /Library/Developer/CommandLineTools/usr/bin
```

Sorpresa: il comando `gcc` risponde *Apple clang*. Su macOS il compilatore è **clang**, un altro compilatore C, che per comodità risponde anche al nome `gcc` e accetta le stesse opzioni. Per tutto ciò che si fa nel corso il comportamento è lo stesso.

Il prompt di macOS

Nel Terminale di macOS il prompt finisce con il simbolo `%` invece che con `$` (per esempio `fabio@MacBook-Air ~ %`). In questa dispensa i comandi sono sempre preceduti da `$`: su Mac si scrivono allo stesso modo.

Linux

Su Ubuntu, Debian e Linux Mint bastano gli stessi due comandi visti per Windows (in Ubuntu, il terminale si apre con Ctrl + Alt + T):

Terminale

```
$ sudo apt update
$ sudo apt install build-essential gdb
```

Su Fedora il comando equivalente è `sudo dnf install gcc make gdb`. La verifica, anche qui, è `gcc --version`.

Passo 3: le estensioni di VS Code

VS Code, appena installato, sa già colorare il codice C. Le **estensioni** sono componenti aggiuntivi gratuiti che gli insegnano a fare di più. Si installano dalla vista *Extensions*: si apre con l'icona dei quattro quadratini nella barra a sinistra (oppure con Ctrl + Shift + X, su Mac Cmd + Shift + X); si scrive il nome nella casella di ricerca e si fa clic su Install (figura 1.8).

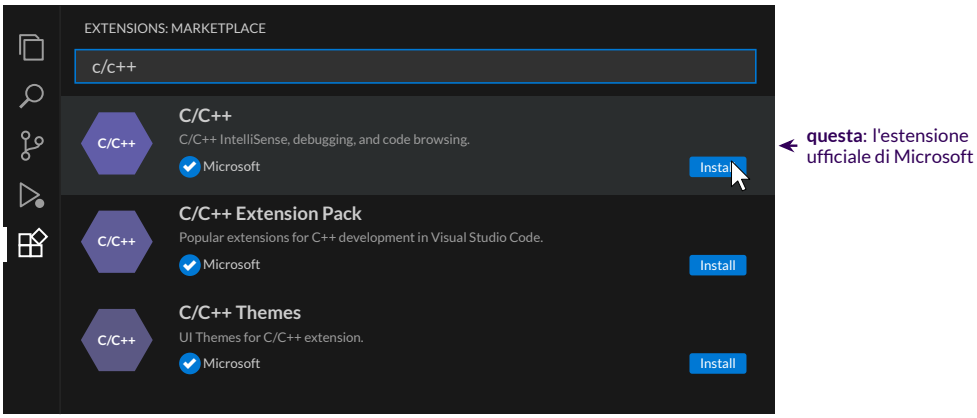


Figura 1.8: *La vista Extensions con la ricerca "c/c++". Basta la prima: C/C++ di Microsoft (si riconosce dal segno blu di "editore verificato").*

Le estensioni da installare sono poche:

Estensione	Per chi	A che cosa serve
C/C++ (Microsoft)	tutti	*IntelliSense*: suggerimenti mentre si scrive (nomi di funzione, parametri), errori sottolineati in rosso ancora prima di compilare, navigazione nel codice (clic destro, *Go to Definition*), debugger.
WSL (Microsoft)	solo Windows	collega VS Code a Ubuntu, come nella figura 1.2.

Spesso non serve nemmeno cercarle: aprendo per la prima volta un file `.c`, VS Code propone da solo, con una notifica in basso a destra, di installare l'estensione consigliata per il linguaggio C.

E la lingua italiana?

VS Code può essere tradotto in italiano con l'estensione *Italian Language Pack*. In questa dispensa però i menu sono citati **in inglese**, come compaiono dopo l'installazione: se scegli l'italiano dovrai tradurre i nomi delle voci (*File* resta *File*, *Terminal* diventa *Terminale*, e così via).

Windows: collegare VS Code a Ubuntu

Su Windows l'ordine conta: prima si installa l'estensione **WSL**, poi si apre VS Code *dentro* Ubuntu, e solo allora si installa **C/C++**. Il modo più semplice per aprire VS Code dentro Ubuntu è farlo dal terminale di Ubuntu, con il comando `code` seguito da un punto (il punto significa "la cartella in cui mi trovo"):

Ubuntu

```
$ code .
Installing VS Code Server for Linux x64 (...)
Downloading: 100%
Unpacking: 100%
```

La prima volta VS Code installa il suo *server* dentro Ubuntu (lo fa da solo, in pochi secondi), poi si apre la finestra. Come si capisce che è collegata a Ubuntu? Dall'**indicatore remoto**, il riquadro blu nell'angolo in basso a sinistra (figura 1.9).



Figura 1.9: *La barra di stato di VS Code collegato a Ubuntu. Se il riquadro in basso a sinistra non dice WSL: Ubuntu, VS Code sta lavorando su Windows e non troverà il compilatore.*

Ora, nella vista *Extensions*, l'estensione C/C++ mostra il pulsante Install in WSL: Ubuntu: è quello giusto, perché l'estensione deve lavorare dove stanno i file e il compilatore. Le volte successive, VS Code ricorda le cartelle aperte: dal menu **File › Open Recent** si riapre direttamente una cartella di Ubuntu, già collegata.

Dove sono i file di Ubuntu?

La cartella personale di Ubuntu (`/home/fabio`) è visibile anche da Esplora file di Windows, alla voce *Linux* nel riquadro di sinistra. Viceversa, da Ubuntu il disco `C:` di Windows si trova in `/mnt/c`. Conviene però tenere i file del corso **dentro Ubuntu**, nella cartella personale: lì compilare è più veloce e non ci sono problemi di permessi.

Passo 4: la cartella del corso

Conviene tenere tutti i programmi del corso in una cartella dedicata. La creiamo dal terminale (su Windows, quello di **Ubuntu**) e la apriamo in VS Code:

Terminale

```
$ mkdir programmazione
$ cd programmazione
$ code .
```

`mkdir` (*make directory*) crea la cartella, `cd` (*change directory*) ci entra, `code .` apre VS Code sulla cartella corrente. In alternativa si può fare tutto con il mouse: si crea la cartella come al solito e la si apre in VS Code da **File › Open Folder...**.

La prima volta che si apre una cartella, VS Code chiede se ci si fida degli autori dei file che contiene (*Do you trust the authors of the files in this folder?*): sono i nostri file, quindi si risponde Yes, I trust the authors. Senza questa conferma VS Code apre la cartella in *modalità con restrizioni* e disattiva le estensioni.

Com'è fatta la finestra di VS Code

La figura 1.10 mostra VS Code con la cartella del corso aperta, un file C nell'editor e il terminale in basso. Vale la pena guardarla con calma: sono le zone che useremo sempre.

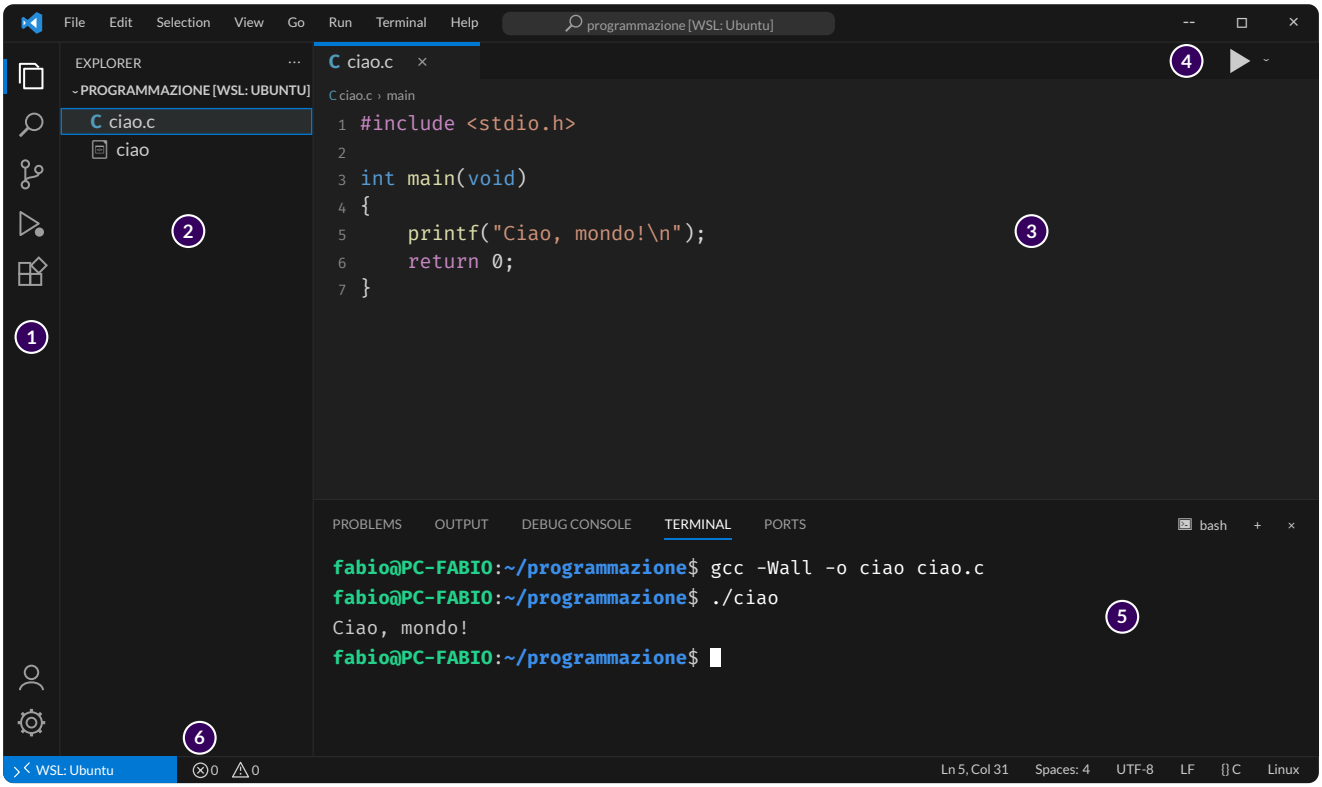


Figura 1.10: *Visual Studio Code con la cartella* `programmazione` *aperta (su Windows, collegato a Ubuntu).*

1. La **barra delle attività**: cambia ciò che compare nella barra laterale. Le icone, dall'alto: *Explorer* (i file), *Search* (cerca nei file), *Source Control* (Git), *Run and Debug* (il debugger), *Extensions*.

2. La **barra laterale**, qui con l'*Explorer*: i file della cartella aperta. Un clic apre un file nell'editor; il pulsante con il foglio e il segno "+", che compare passando con il mouse sul nome della cartella, crea un nuovo file.

3. L'**editor**: il file aperto, con i numeri di riga a sinistra. In alto ci sono le schede dei file aperti e il "percorso" (*ciao.c* › *main*: il cursore è dentro la funzione `main`).

4. Il pulsante *Run*, aggiunto dall'estensione C/C++ (ne parleremo alla fine del capitolo).

5. Il **pannello** in basso, qui con il *terminale* integrato: un vero terminale (su Windows, di Ubuntu) già posizionato nella cartella aperta. Si apre dal menu **Terminal › New Terminal**; accanto alla voce del menu è indicata anche la scorciatoia da tastiera, che dipende dalla tastiera in uso.

6. La **barra di stato**: a sinistra l'indicatore remoto e il numero di errori e avvisi; a destra la posizione del cursore (*Ln* riga, *Col* colonna) e il linguaggio riconosciuto (*C*).

Creare un file

Un nuovo file si crea dall'*Explorer* (pulsante "nuovo file" accanto al nome della cartella) oppure da **File** › **New File...**. Il nome deve finire con `.c`: è l'**estensione** del nome a dire a VS Code (e a gcc) che il file contiene codice C. Con un nome come `ciao.txt`, o senza estensione, niente colori, niente suggerimenti e gcc si rifiuterebbe di compilarlo.

Nomi di file semplici

Usa nomi brevi, in minuscolo, **senza spazi** e senza lettere accentate: `esercizio1.c`, `media_voti.c`. Gli spazi nei nomi costringono a usare le virgolette nei comandi del terminale e sono una fonte continua di errori.

IL TERMINALE

Il terminale è un dialogo scritto con il computer: noi scriviamo un **comando** e premiamo Invio, il computer lo esegue e risponde con del testo. Il programma che legge i comandi si chiama **shell**; in Ubuntu è *bash*, su macOS *zsh*. Per il corso servono pochi comandi, ma vanno capiti bene: sono il modo in cui compileremo ed eseguiremo ogni programma.

Il prompt

Quando la shell è pronta mostra il **prompt**, una riga che finisce con `$` (su Mac `%`) e dopo la quale si scrive. Il prompt di Ubuntu non è decorativo: dice *chi* siamo, *dove* siamo e che la shell *aspetta* un comando (figura 1.11).



Figura 1.11: *Le parti del prompt di Ubuntu.*

La parte più importante è la **cartella corrente** (o *directory di lavoro*): ogni comando lavora *lì*, a meno di indicare diversamente. Se scriviamo `gcc ciao.c` e il file `ciao.c` non sta nella cartella corrente, gcc non lo trova. Moltissimi problemi dei principianti si risolvono guardando il prompt.

Il prompt, nella dispensa

D'ora in poi i comandi sono mostrati con il solo `$` come prompt e in **grassetto bianco**; le righe normali sono ciò che il computer risponde, e il testo in giallo è ciò che si scrive mentre un programma è in esecuzione. Il `$` **non va digitato**: si scrive solo ciò che lo segue.

Cartelle e percorsi

I file sono organizzati in cartelle, che contengono altre cartelle, che contengono altri file: un **albero**, che in Linux parte da un'unica radice chiamata `/` (figura 1.12).

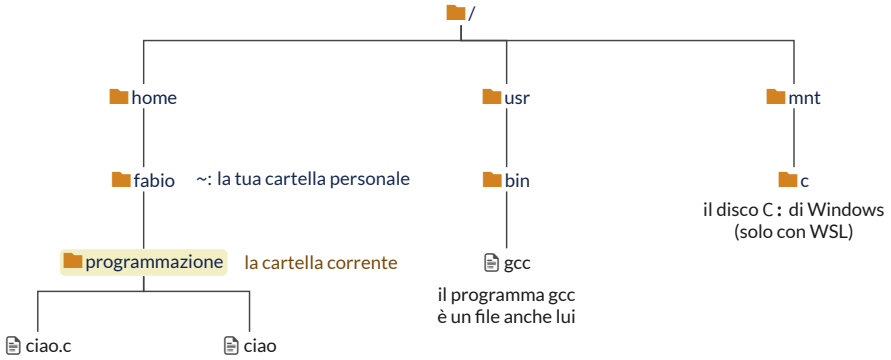


Figura 1.12: *Una parte dell'albero delle cartelle di Ubuntu.*

Un file si indica con il suo **percorso**, cioè la strada per raggiungerlo, con i nomi delle cartelle separati da `/`:

- un percorso **assoluto** parte dalla radice e vale ovunque ci si trovi: `/home/fabio/programmazione/ciao.c`;
- un percorso **relativo** parte dalla cartella corrente: se si è in `/home/fabio`, lo stesso file è `programmazione/ciao.c`; se si è già in `programmazione`, è semplicemente `ciao.c`.

Tre nomi speciali completano il quadro: `.` è la cartella corrente, `..` è la cartella che la contiene ("sali di un livello") e `~` è la cartella personale. Dalla cartella `programmazione`, quindi, `..` è `/home/fabio`.

Comando	Che cosa fa	Da dove viene il nome
`pwd`	stampa il percorso della cartella corrente	*print working directory*
`ls`	elenca il contenuto della cartella corrente	*list*
`ls -l`	come sopra, con dimensione e data dei file	*long*
`cd nome`	entra nella cartella `nome`	*change directory*
`cd ..`	sale nella cartella superiore	
`cd`	torna nella cartella personale	
`mkdir nome`	crea la cartella `nome`	*make directory*
`cp da a`	copia il file `da` nel file `a`	*copy*
`mv da a`	sposta o rinomina un file	*move*
`rm file`	cancella un file (**senza cestino!**)	*remove*
`cat file`	mostra il contenuto di un file di testo	*concatenate*
`clear`	pulisce lo schermo del terminale	

Ecco una breve sessione di lavoro; la spiegazione segue subito dopo.

Terminale

```
$ pwd
/home/fabio
$ ls
programmazione
$ cd programmazione
$ ls
ciao.c
$ mkdir lezione2
$ ls
ciao.c  lezione2
$ cp ciao.c lezione2/prova.c
$ cd lezione2
$ ls
prova.c
$ cd ..
$ pwd
/home/fabio/programmazione
```

Si parte dalla cartella personale (`pwd` lo conferma), si entra in `programmazione`, si crea la sottocartella `lezione2` e ci si copia il file `ciao.c` con il nuovo nome `prova.c` (si noti il percorso relativo `lezione2/prova.c`). Poi si entra in `lezione2`, si controlla che la copia ci sia, e con `cd ..` si torna su di un livello. Molti comandi, quando riescono, **non stampano niente**: è la convenzione di Linux, "nessuna notizia, buona notizia".

`rm` non perdona

Il comando `rm` cancella definitivamente: il file non passa per il cestino e non si recupera. Controlla sempre due volte il nome prima di premere Invio.

Trucchi che fanno risparmiare tempo

- Tab **completa** i nomi: scrivendo `cd prog` e premendo Tab, la shell completa da sola in `cd programmazione/`. Oltre a risparmiare tempo evita gli errori di battitura: se il completamento non funziona, il file non si trova dove pensiamo.
- ↑ e ↓ scorrono i **comandi già dati**: per ricompilare dopo una modifica basta premere ↑ fino a ritrovare il comando `gcc` e poi Invio.
- Ctrl + C **interrompe** il programma in esecuzione. Se un programma non termina mai (capiterà, con i cicli infiniti del capitolo **??**), è il modo per fermarlo.
- Ctrl + D, scritto come input a un programma, segnala la **fine dell'input**; lo useremo dal capitolo **??**.
- Nel terminale Ctrl + C non copia: per copiare e incollare, nel terminale di VS Code si usano il tasto destro del mouse o Ctrl + Shift + C e Ctrl + Shift + V.

IL PRIMO PROGRAMMA

È il momento di mettere insieme i pezzi. Nella cartella `programmazione` crea il file `ciao.c` e scrivici, esattamente così, il programma seguente.

```c
#include <stdio.h>

int main(void)
{
    printf("Ciao, mondo!\n");
    return 0;
}
```

Il significato di ogni riga sarà spiegato nel capitolo **??**; per ora basta sapere che `printf` stampa il testo fra virgolette, e che `\n` significa "vai a capo". Prima di proseguire, **salva** il file con Ctrl + S (su Mac Cmd + S).



un pallino: modifiche **non salvate** — una crocetta: il file è **salvato**

Figura 1.13: *La scheda del file dice se ci sono modifiche non salvate.*

gcc legge il file **dal disco**: se le modifiche non sono salvate, compila la versione vecchia. È l'errore più frequente all'inizio ("ho corretto ma non cambia niente!"). Per evitarlo una volta per tutte, si può attivare il salvataggio automatico dal menu **File › Auto Save**.

Compilare ed eseguire

Ora, nel terminale, compiliamo ed eseguiamo:

```
Terminale

$ gcc ciao.c
$ ls
a.out  ciao.c
$ ./a.out
Ciao, mondo!
```

- `gcc ciao.c` compila il file. Non stampa nulla: nessun messaggio significa **nessun errore**. Il risultato è un nuovo file, l'eseguibile, che per tradizione si chiama `a.out`.
- `./a.out` esegue il programma, che stampa la sua riga.

Il nome `a.out` va bene per una prova, ma con più programmi nella stessa cartella ogni compilazione sovrascriverebbe la precedente. Con l'opzione `-o` (*output*) si sceglie il nome dell'eseguibile; e con `-Wall` (*warnings: all*) si chiede a gcc di segnalare anche le cose sospette, che non impediscono la compilazione ma spesso nascondono un errore. È il modo in cui conviene compilare sempre:

```
Terminale

$ gcc -Wall -o ciao ciao.c
$ ./ciao
Ciao, mondo!
```

Ogni volta che si cambia il programma bisogna ripetere **tutto il giro** (figura 1.14): l'eseguibile è una traduzione fatta in un certo momento, e non si aggiorna da solo quando cambia il testo.

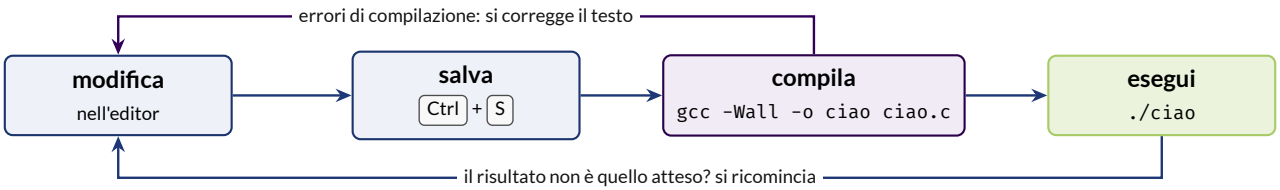


Figura 1.14: *Il ciclo di lavoro: modifica, salva, compila, esegui.*

Perché `./`?

Perché per eseguire il programma si scrive `./ciao` e non solo `ciao`, mentre per gcc basta `gcc`? Quando si scrive il nome di un comando, la shell non lo cerca dappertutto: lo cerca solo in un elenco di cartelle prestabilite, quelle dove stanno i programmi installati (come `/usr/bin`, dove si trova `gcc`). L'elenco è una *variabile d'ambiente* chiamata `PATH`, e per sicurezza non contiene la cartella corrente. Scrivendo `./ciao` si dà il **percorso** del file invece del nome di un comando: "il programma `ciao` che sta *qui*" (`.` è la cartella corrente).

Terminale

```
$ ciao
ciao: command not found
$ ./ciao
Ciao, mondo!
```

Gli errori di compilazione

Che cosa succede se il programma contiene un errore? Togliamo il punto e virgola alla fine della riga di `printf` e ricompiliamo:

errore.c

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     printf("Ciao, mondo!\n")
6     return 0;
7 }
```

Terminale

```
$ gcc -Wall -o errore errore.c
errore.c: In function ‘main’:
errore.c:5:29: error: expected ‘;’ before ‘return’
    5 |     printf("Ciao, mondo!\n")
      |                             ^
      |                             ;
    6 |     return 0;
      |     ~~~~~~
```

Il messaggio va letto con calma, perché dice quasi tutto (figura 1.15): **dove** è l'errore (file, riga e colonna), **di che tipo** è e **che cosa** si aspettava il compilatore. Le righe successive ricopiano il codice e indicano con `^` il punto esatto; gcc arriva perfino a suggerire il `;` mancante. Quando c'è un `error` l'eseguibile **non viene prodotto**: bisogna correggere e ricompilare.



Figura 1.15: *Anatomia di un messaggio di errore di gcc.*

Si noti che l'errore è segnalato alla riga **5**, alla fine della riga di `printf`, anche se il compilatore se ne accorge solo leggendo `return` alla riga 6: il punto e virgola poteva mancare solo lì. In generale l'errore sta nella riga indicata **o poco prima**.

Esistono due tipi di messaggi:

- un `error` è un errore vero: il testo non è C valido e l'eseguibile non viene creato;
- un `warning` (avviso) segnala qualcosa di lecito ma sospetto: l'eseguibile viene creato, ma quasi sempre il warning indica un errore di ragionamento. **I warning vanno trattati come errori**: un programma compilato con `-Wall` non deve produrne nessuno.

Leggere solo il primo errore

Un solo errore può confondere il compilatore e generarne a cascata molti altri, alcuni dei quali senza senso. La regola pratica: si corregge **il primo** messaggio dell'elenco, si ricompila, e si ripete.

Con l'estensione C/C++ installata, VS Code si accorge di molti errori **prima** della compilazione e li sottolinea con un'ondina rossa; passando con il mouse sopra la sottolineatura compare la spiegazione (figura 1.16). Il numero di errori compare anche nella barra di stato, accanto a ⊗.



Figura 1.16: *L'estensione C/C++ sottolinea l'errore mentre si scrive.*

E il pulsante "Run"?

L'estensione C/C++ aggiunge in alto a destra nell'editor un pulsante ▶ (*Run C/C++ File*) che compila ed esegue il file aperto con un clic; la prima volta chiede quale compilatore usare (si sceglie *gcc*). È comodo, ma in questa dispensa compileremo **sempre dal terminale**: è ciò che si fa a lezione e all'esame, permette di scegliere le opzioni di gcc, e soprattutto fa vedere che cosa succede davvero, passo per passo. Dal capitolo **??** useremo `make`, che si lancia anch'esso dal terminale.

Quando qualcosa non funziona

Sintomo	Causa probabile e rimedio
`gcc: command not found`	Il compilatore non è installato (ripeti il passo 2). Su Windows, controlla di essere nel terminale di **Ubuntu** (prompt con `$`) e non in PowerShell (prompt `PS C:\...`).
`code: command not found`	Su Windows chiudi e riapri il terminale di Ubuntu dopo aver installato VS Code (il `PATH` si aggiorna solo all'apertura); su Mac esegui *Shell Command: Install 'code' command in PATH*.
`ciao.c: No such file or directory`	Il file non è nella cartella corrente: controlla con `pwd` e `ls` dove ti trovi e che cosa c'è, oppure il file non è stato salvato con quel nome.
`./ciao: No such file or directory`	Il programma non è stato compilato, oppure la compilazione è fallita (c'erano degli `error`), oppure l'eseguibile ha un altro nome (`a.out`?).
Ho corretto ma il programma fa ancora la cosa sbagliata	Hai salvato? Hai ricompilato? Stai eseguendo l'eseguibile giusto?
Il programma non termina	Probabilmente un ciclo infinito, oppure il programma aspetta un input da tastiera. Ctrl + C lo interrompe.
In VS Code `#include <stdio.h>` è sottolineato in rosso	L'estensione C/C++ non trova il compilatore. Su Windows: VS Code non è collegato a Ubuntu (guarda l'indicatore remoto) o l'estensione non è installata *in WSL*. Altrove: verifica che `gcc --version` funzioni.

In sintesi

Per scrivere ed eseguire un programma C servono tre strumenti: un **editor** (VS Code) per scrivere il testo del programma in un file `.c`, un **compilatore** (gcc) che lo traduce in linguaggio macchina, e un **terminale** da cui lanciare il compilatore e il programma. Su Windows tutto ciò vive in Ubuntu, grazie a WSL. Il ciclo di lavoro è sempre lo stesso: *modifica, salva, compila, esegui*; e i messaggi di gcc indicano file, riga e colonna del problema.

Riferimenti

- **Installare Linux in Windows con WSL** — Microsoft Learn.
  `https://learn.microsoft.com/it-it/windows/wsl/install`
- **Using C++ and WSL in VS Code** — documentazione di VS Code (vale anche per il C).
  `https://code.visualstudio.com/docs/cpp/config-wsl`
- **Using Clang in Visual Studio Code** (macOS) e **Using C++ on Linux in VS Code** — documentazione di VS Code.
  `https://code.visualstudio.com/docs/cpp/config-clang-mac`
  `https://code.visualstudio.com/docs/cpp/config-linux`
- **User Interface** — la guida alle zone della finestra di VS Code.
  `https://code.visualstudio.com/docs/getstarted/userinterface`